Informe Final de Estrategia de Testing
Módulo de Persistencia y Dominio Clínico

Grupo 3

Integrantes: Archetti Lestarpe, Santiago - Cherepakhin, Victoria - Fossati, Facundo - Milosevic, Natasha Aylen

Índice

Parte 1: Estrategia y Pruebas Atómicas (AsociadoDTO) 2

1. Introducción y Marco de Requisitos 2
2. Diseño Formal de Casos de Prueba (Caja Negra) 3
3. Diagnóstico de Defectos y Hallazgos (Ejecución en rojo) 5
4. Refactorización del Código y Validación Dinámica (Ejecución en verde) 8

Parte 2: Integración de Persistencia (DAO y BaseDeDatos) 11

1. Componentes Bajo Prueba y Arquitectura de Persistencia 11
2. Especificación de Requisitos de Integración (SRS 3.2 y 3.3) 11
3. Entorno de Ejecución y Herramientas de Integración 12
4. Diseños de Casos de Prueba de Integración 12
5. Diagnóstico de Defectos y Hallazgos de Integración (Ejecución en ROJO) 15
6. Refactorización y Resultados en VERDE 17

Parte 3: Módulo de Ambulancia (State, Observer y Concurrencia) 18

1. Introducción y Marco de Requisitos 18
2. Diseño Formal de Casos de Prueba 19
3. Diagnóstico de Defectos y Hallazgos (Ejecución en ROJO) 21
4. Refactorización Realizada 22
5. Resultados tras la Refactorización (Ejecución en VERDE) 23

Conclusiones 24

Apéndices 26

Parte 1: Trazas de Pruebas Atómicas (AsociadoDTO) 26

Parte 2: Trazas de Ejecución de Pruebas de Integración 27

Parte 3: Evidencias de Ejecución 28

Parte 1

1. Introducción y Marco de Requisitos

1.1. Contexto de la Aplicación Clínica y Arquitectura del Sistema

El presente informe documenta el desarrollo y la ejecución de la estrategia de testing para el sistema de gestión de una clínica médica privada, desarrollado bajo la arquitectura Modelo-Vista-Controlador (MVC) y guiado por principios de diseño orientado a objetos. El núcleo del sistema comprende módulos de facturación, gestión de pacientes y médicos, simulación de sala de espera y ambulancia concurrente, y una capa de persistencia relacional sustentada en un motor MySQL mediante conectividad JDBC.

1.2. Componente Bajo Prueba: AsociadoDTO (Análisis Atómico)

La clase AsociadoDTO, ubicada en el paquete persistencia, constituye el objeto de transferencia de datos (Data Transfer Object) encargado de transportar la información de los asociados entre la interfaz gráfica de usuario (VentanaPrincipal), el controlador del sistema, la clase fachada Clinica y el objeto de acceso a datos (Data Access Object).

Como objeto de transferencia puro (POJO), AsociadoDTO implementa la interfaz Serializable y carece de dependencias hacia clases del dominio, servicios de red o motores de base de datos. Esta naturaleza atómica permite someter la clase a un proceso de verificación dinámico en estricto aislamiento, garantizando que los datos ingresados cumplan con los contratos de integridad estructural sin necesidad de levantar infraestructura externa.

La estructura interna del DTO encapsula siete atributos privados:

- ★ Seis atributos de tipo String: nombre, apellido, dni, ciudad, calle y telefono.
- ★ Un atributo de tipo entero primitivo (int): numero.

La clase expone un constructor por defecto, un constructor parametrizado que acepta los siete campos y los correspondientes métodos de acceso y modificación (*getters* y *setters*).

1.3. Especificación Formal de Requisitos (SRS 3.1) y Criterio de Verdad

La especificación formal de requisitos del módulo de persistencia (SRS_Persistencia_Clinica, sección 3.1) establece el conjunto de reglas contractuales e invariantes de datos que el contenedor AsociadoDTO debe hacer cumplir en todo su ciclo de vida.

Las reglas de validación establecidas en la sección 3.1 de la SRS se sintetizan a continuación:

Atributo	**Tipo de Dato**	**Regla de Validación Estructural (SRS 3.1)**
nombre	String	No puede ser nulo (nombre != null).
apellido	String	No puede ser nulo (apellido != null).
dni	String	No puede ser nulo (dni != null); actúa como identificador único.

ciudad	String	No puede ser nula (ciudad != null).
calle	String	No puede ser nula (calle != null).
numero	int	Debe ser mayor o igual a 0 (numero >= 0).
telefono	String	No puede ser nulo (telefono != null).

1.4. Entorno de Ejecución y Herramientas de Testing

La automatización de las pruebas unitarias se configuró sobre un entorno estandarizado de construcción e integración continua, asegurando la reproducibilidad total de los experimentos dinámicos:

- ★ *Framework de Testing:* JUnit versión 4.13.2.
- ★ *Gestor de Construcción:* Apache Maven 3.9.9.
- ★ *Plugin de Ejecución de Pruebas:* maven-surefire-plugin versión 3.2.5.
- ★ *Habilitación de Aserciones Java:* Configurada explícitamente en el archivo de construcción mediante <enableAssertions>true</enableAssertions> (equivalente a la bandera -ea de la JVM) para activar la evaluación de la sentencia assert durante el ciclo del motor Surefire.
- ★ *Entorno de Compilación:* JDK 17.0.2 configurado con compatibilidad de lenguaje destino Java 8 (source y target 8).
- ★ *Ubicación de la Suite:* src/test/java/persistencia/AsociadoDTOTest.java.

2. Diseño Formal de Casos de Prueba (Caja Negra)

2.1. Tabla de Escenarios de Prueba

Dado que las pruebas de la clase AsociadoDTO deben comprobar el comportamiento atómico del componente sin interacción con recursos persistentes ni servicios concurrentes, se define un escenario único centrado en el ciclo de vida del objeto en memoria.

Nro.de Escenario	**Descripción**
1	Objeto de la clase AsociadoDTO instanciado en memoria en estado atómico, aislado de bases de datos relacionales, hilos de simulación o componentes visuales.

2.2. Particiones de Equivalencia y Análisis de Valores Límite

Se aplicó la técnica de partición en clases de equivalencia para dividir el dominio de entrada de cada parámetro en subconjuntos con comportamiento homogéneo, complementada con un análisis de valores límite en los bordes críticos de decisión.

Para los seis atributos de tipo String, la especificación exige estrictamente la condición de no nulidad. Los valores de cadenas vacías ("") o compuestas por espacios en blanco no son restringidos por el DTO según la SRS, permaneciendo dentro del alcance de validación de la

interfaz gráfica (VentanaPrincipal). Para el atributo entero número, el límite exacto entre la clase válida e inválida se ubica en cero (0).

Atributo / Condición	**Clase Válida**	**Clase Inválida**	**Valores Límite Analizados**
Atributos String(6 atributos)	Cadena de texto no nula (ej. "Juan","Pérez", "12345678")	Referencia nula (null)	La cadena vacía ("") se clasifica fuera del alcance del DTO por exigirse únicamente la no nulidad en la SRS.
Atributo número	Número entero mayor o igual a 0 (numero >= 0, ej. 1000)	Número entero negativo (numero < 0)	-1 (Valor límite inválido superior) 0 (Valor límite válido inferior)

2.3. Matriz y Batería de Casos de Prueba Implementados

Con el fin de garantizar la trazabilidad entre el diseño teórico y el código ejecutable, la suite AsociadoDTOTest.java formaliza cinco métodos de prueba automatizados en JUnit 4.

La batería contempla la verificación de asignaciones válidas, la exploración del valor límite exacto, la evaluación aislada de la nulidad en los seis *setters* y la resistencia de los constructores ante combinaciones de datos corruptos.

N.º	**Método de Prueba (@Test)**	**Tipo de Clase**	**Datos de Entrada Proporcionados**	**Resultado Esperado según Contrato SRS**
1	testAsignacionYRecuperacionValida	Válida	nombre="Juan" apellido="Pérez" dni="12345678" ciudad="Mar del Plata" calle="San Martín" numero=1000 telefono="223456789"	Asignación exitosa. Cada método *getter* retorna exactamente la referencia o valor primitivo asignado.
2	testNumeroValorLimiteCeroValido	Válida (Valor limite)	Invocación a setNumero(0)	Asignación aceptada. El método getNumero() retorna 0 sin lanzar

				excepciones ni errores.
3	testSettersInvaidosNulos	Inválida	Invocación individualizada de null en setNombre, setApellido, setDni, setCiudad, setCalle y setTelefono	Cada método *setter* detecta la condición nula y rechaza la operación lanzando IllegalArgumentException.
4	testSetNumeroInvalidoNegativo	Inválido (Valor Límite)	Invocación a setNumero(-1)	Operación rechazada. Se dispara una excepción IllegalArgumentException.
5	testConstructorParametrizadoValidacion	Inválidas	Instanciación directa: new AsociadoDTO(null, null, null, null, null, -5, null)	El constructor intercepta los parámetros inválidos y rechaza la creación lanzando IllegalArgumentException.

3. Diagnóstico de Defectos y Hallazgos (Ejecución en rojo)

3.1. Evaluación Inicial sobre Código Provisto y Resultado BUILD FAILURE

La suite de pruebas automatizada se ejecutó en primera instancia sobre el código fuente original sin modificar lo extraído del paquete distribuido. Con la bandera de aserciones de Java activada en Surefire (-ea), el motor de pruebas ejecutó la suite obteniendo un fallo de construcción (BUILD FAILURE).

La ejecución sobre el código fuente original arrojó el siguiente resumen cuantitativo:

```
[ERROR] Tests run: 5, Failures: 2, Errors: 0, Skipped: 0 <<< FAILURE!
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
```

La evaluación individualizada de los métodos de prueba sobre la versión original se detalla en la siguiente tabla de diagnóstico:

Método @Test	**Estado de Ejecución**	**Defecto Asociado en el Código Provisto**

testAsignacionYRecuperacionValida	Aprobado	Ninguno. Los *setters* y *getters* funcionan para valores positivos y cadenas válidas.
testNumeroValorLimiteCeroValido	Fallo	**Defecto D1:** Rechazo del valor límite válido numero = 0 mediante operador estricto.
testSettersInvalidosNulos	Aprobado (Condicionado)	**Defecto D3:** Se aprueba únicamente por presencia de -ea, lanzando AssertionError.
testSetNumeroInvalidoNegativo	Aprobado (Condicionado)	**Defecto D3:** Se aprueba únicamente por presencia de -ea, lanzando AssertionError.
testConstructorParametrizadoValidacion	Fallo	**Defecto D2:** Omisión completa de validación en el constructor parametrizado.

3.2. Diagnóstico Detallado del Defecto

D1: Rechazo de numero = 0 (Error Off-by-One)

- ★ Ubicación en Código Fuente → Clase AsociadoDTO, método setNumero(int numero), línea 87.
- ★ Sentencia Provista Original → assert numero > 0;
- ★ Especificación Requerida (SRS 3.1) → numero >= 0.
- ★ Síntoma Observado → Al invocar setNumero(0), la ejecución interrumpe abruptamente arrojando la excepción java.lang.AssertionError.
- ★ Causa Raíz → El desarrollador utilizó un operador de desigualdad estricta (>) en lugar de permitir la inclusión del cero (>=). Este constituye un error clásico de análisis de valores límite (*off-by-one error*).
- ★ Severidad → Media-Alta. Se bloquea el registro legítimo de asociados domiciliados en la numeración catastral cero.

D*2: Constructor Parametrizado Inconsistente sin Validaciones*

- ★ Ubicación en Código Fuente → Clase AsociadoDTO, constructor parametrizado AsociadoDTO(String, String, String, String, String, int, String).
- ★ Sentencia Provista Original → Asignación directa de argumentos a campos de instancia (this.nombre = nombre; ... this.numero = numero;).
- ★ Síntoma Observado → La llamada new AsociadoDTO(null, null, null, null, null, -5, null) finaliza exitosamente instanciando un objeto en memoria cuyos campos violan la totalidad de los contratos de la SRS.
- ★ Causa Raíz → Las validaciones defensivas fueron codificadas exclusivamente en el cuerpo de los métodos *setters*. El constructor parametrizado omite invocar las

validaciones o delegar la asignación en los *setters*, creando una vía alternativa de elusión de las reglas de negocio.

★ Severidad → Alta. Permite la inyección de objetos DTO corruptos hacia la capa DAO y la base de datos MySQL.

D*3: Fragilidad de la Validación con Sentencias assert*

★ Ubicación en Código Fuente → Todos los métodos *setters* de la clase AsociadoDTO.

★ Sentencia Provista Original → Uso de la palabra reservada assert (ej. assert nombre != null;).

★ Síntoma y Problema Mecánico → Las aserciones nativas de Java están desactivadas por defecto durante el tiempo de ejecución normal de la Máquina Virtual de Java. Sin la bandera -ea en producción, los *setters* aceptan silenciosamente referencias nulas y números negativos.

★ Análisis de Diseño → La sentencia assert produce un AssertionError, el cual hereda de java.lang.Error y no de java.lang.Exception. Un error de la JVM representa una condición catastrófica no recuperable y no debe ser utilizado para señalar violaciones de argumentos en APIs o contratos de transferencia.

★ Severidad → Alta. La integridad de la persistencia de datos queda condicionada a la configuración externa de la JVM.

3.3.Hallazgos Complementarios e Incompatibilidades de Compilación

Durante el proceso de auditoría y ejecución de la prueba atómica se identificaron inconsistencias adicionales en el código del proyecto que, si bien no pertenecen a la lógica interna del DTO, impactan en la compilación y mantenibilidad del sistema:

★ Javadoc Desactualizado
- El comentario cabecera de AsociadoDTO mencionaba la existencia de un atributo id.
- Inconsistencia de documentación. AsociadoDTO utiliza el dni como identificador único.

★ Falta de serialVersionUID
- AsociadoDTO implementa Serializable pero no declara la constante de versión de serialización.
- Genera advertencias de compilador y riesgos de deserialización entre versiones de la JVM.

★ Discrepancia DTO vs. Vista
- La clase VentanaPrincipal rechaza numero <= 0 y cadenas vacías mediante datosAsociadoDTOIncorrectoException.
- Muestra que la interfaz gráfica aplica reglas más restrictivas que la especificación de persistencia SRS 3.1.

4. Refactorización del Código y Validación Dinámica (Ejecución en verde)

4.1 Estrategia de Refactorización y Métodos Auxiliares de Validación

Para resolver los defectos D1, D2 y D3 sin alterar la firma pública ni romper la compatibilidad con el resto de los módulos de la clínica, se refactorizó la clase AsociadoDTO reemplazando el uso de assert por excepciones explícitas de tipo IllegalArgumentException.

Las validaciones se centralizaron en dos métodos estáticos privados de utilidad dentro del DTO, garantizando reutilización y coherencia entre los constructores y los métodos de modificación:

```
private static String noNulo(String valor, String campo) {
   if (valor == null) {
        throw new IllegalArgumentException("El campo " + campo + " no puede ser
nulo.");
   }
   return valor;
}
private static int numeroValido(int numero) {
   if (numero < 0) {
        throw new IllegalArgumentException("El numero debe ser mayor o igual a 0.
Valor ingresado: " + numero);
   }
   return numero;
}
```

La aplicación de estos métodos auxiliares en los *setters* y en el constructor parametrizado asegura que ningún objeto AsociadoDTO pueda ser instanciado o modificado violando las reglas contractuales:

```
public void setNumero(int numero) {
   this.numero = numeroValido(numero);
}
public AsociadoDTO(String nombre, String apellido, String dni, String ciudad,
                String calle, int numero, String telefono) {
   this.nombre = noNulo(nombre, "nombre");
   this.apellido = noNulo(apellido, "apellido");
   this.dni = noNulo(dni, "dni");
   this.ciudad = noNulo(ciudad, "ciudad");
   this.calle = noNulo(calle, "calle");
   this.numero = numeroValido(numero);
   this.telefono = noNulo(telefono, "telefono");
}
```

El constructor por defecto se conservó sin validación para permitir la instanciación limpia requerida por el marco de mapeo del DataAccessObject, el cual crea el contenedor en blanco

y posteriormente invoca los *setters* validados. Adicionalmente, se incorporó la constante private static final long serialVersionUID = 1L; y se depuró el Javadoc.

4.2 Verificación de Reejecución de la Suite JUnit y Resultado BUILD SUCCESS
Tras aplicar la refactorización en AsociadoDTO.java, se procedió a reejecutar la suite de pruebas unitarias AsociadoDTOTest mediante Maven.
La traza arrojada por la consola confirmó el paso exitoso de la totalidad de las pruebas unitarias:

```
[INFO] Running persistencia.AsociadoDTOTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in persistencia.AsociadoDTOTest
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

La matriz final de resultados demuestra la resolución de las fallas previamente detectadas:

Método de Prueba (@Test)	**Estado Post-Refactorización**	**Comportamiento Verificado**
testAsignacionYRecuperacionValida	Aprobado	Preservación de la integridad funcional de asignación y lectura.
testNumeroValorLimiteCeroValido	Aprobado	Aceptación correcta del valor límite inferior numero = 0 (Defecto D1 resuelto).
testSettersInvaidosNulos	Aprobado	Intercepción de nulos mediante IllegalArgumentException independiente de -ea.
testSetNumeroInvalidoNegativo	Aprobado	Rechazo de enteros negativos mediante IllegalArgumentException (Defecto D3 resuelto).
testConstructorParametrizadoValidacion	Aprobado	Intercepción de datos nulos y negativos en instanciación (Defecto D2 resuelto).

4.3 Análisis de Impacto en los Módulos de Persistencia (DataAccessObject y BaseDeDatos)
La transformación de las aserciones en excepciones de tipo IllegalArgumentException fortalece la arquitectura global del módulo de persistencia.

Cuando el DataAccessObject ejecute operaciones de lectura desde la base de datos MySQL mediante el método CrearDTO, cualquier corrupción de datos a nivel de tabla (como un valor NULL inesperado en columnas obligatorias) será interceptada inmediatamente por el DTO como una excepción de contrato administrable, evitando la propagación de referencias nulas o errores no controlados hacia la capa de negocio.

Parte 2

1. Componentes Bajo Prueba y Arquitectura de Persistencia

Aquí evaluamos la integración dinámica y la comunicación real entre los tres componentes de la capa de persistencia interactuando contra un motor de base de datos relacional (MySQL / MariaDB en localhost:3306), prescindiendo de dobles de prueba:

- ★ *AsociadoDTO:* Incorpora las validaciones de integridad atómicas refactorizadas en la Parte 1 y actúa como contenedor de transporte entre el DAO y las sentencias SQL.
- ★ *DataAccessObject (DAO):* Responsable de traducir las operaciones de negocio (guardar, cargar, cargarBD, cargarConLimite, actualizar, eliminar) a sentencias SQL y mapear cada registro resultante a instancias DTO mediante CrearDTO.
- ★ *BaseDeDatos:* Componente administrativo que aplica el patrón de diseño **Singleton** para garantizar una única instancia de conexión JDBC durante todo el ciclo de vida de la aplicación. Al inicializarse, ejecuta la rutina iniciarBD() para crear el esquema, descartar y recrear la tabla Asociados y poblar con 20 registros semilla de prueba. Administra la ejecución de consultas con PreparedStatement.

2. Especificación de Requisitos de Integración (SRS 3.2 y 3.3)

2.1. Conexión y Administración (SRS 3.2)

- ★ *Singleton Estricto:* La clase BaseDeDatos debe restringir su instanciación a un único manejador de conexión activo durante la ejecución.
- ★ *Inicialización Automática (iniciarBD()):* Al arrancar el sistema o reiniciar el entorno de pruebas, debe verificar/crear el esquema objetivo, eliminar y recrear la tabla Asociados, y cargar los 20 registros semilla predefinidos.
- ★ *Sentencias Parametrizadas:* Todas las consultas, inserciones, actualizaciones y eliminaciones deben utilizar declaraciones preparadas (PreparedStatement) para prevenir vulnerabilidades de inyección SQL.

2.2. Acceso a Datos (SRS 3.3)

- ★ *Operación de Guardado:* Debe rechazar entidades nulas, convertirlas a DTO e insertar el registro persistente.
- ★ *Operaciones de Carga:* Permitir la recuperación por identificador único, por lotes acotados (límite) o la colección completa desde la base de datos.
- ★ *Actualización y Eliminación:* El criterio único de búsqueda y modificación debe ser el número de documento (**DNI**).

3. Entorno de Ejecución y Herramientas de Integración

- ★ *Framework de Testing:* JUnit 4.13.2.
- ★ *Gestor de Construcción:* Maven 3.9.9 con maven-surefire-plugin 3.2.5 (aserciones habilitadas -ea).
- ★ *Driver JDBC:* com.mysql:mysql-connector-j:8.0.33 (incorporado al pom.xml tras detectar su ausencia en el proyecto original).
- ★ *Motor Relacional:* MariaDB 10.4.24 (servidor MySQL distribuido en XAMPP) sobre localhost:3306.
- ★ *Credenciales de Conexión:* Usuario progra_c / Contraseña progra_c sobre el esquema.
- ★ *Aislamiento de Pruebas:* El método @Before de la suite PersistenciaIntegracionTest.java invoca BaseDeDatos.getInstance().iniciarBD() antes de cada prueba. Esto elimina la tabla, la vuelve a crear y recarga las 20 filas semilla,

garantizando que cada prueba se ejecute en un estado limpio e independiente sin contaminación lateral.

4. Diseños de Casos de Prueba de Integración

4.1. Particiones de Equivalencia y Valores Límite de Integración

Entrada	**Clase Válida**	**Clase Inválida**	**Valores Límite Analizados**
Identificador (id)	Existe en los datos semilla	No existe en la base de datos	1 (Primer registro semilla: Juan State) 999 (ID fuera del conjunto)
Límite de Carga (limite)	1 \le n \le 20	Rechazado por precondición en DAO	1 (Mínimo útil) 5 (Interior) 50 (Por encima del total de filas)
Documento (dni)	Existe en la BD, o es nuevo y único	Duplicado, inexistente o nulo	12345678 (Juan - Fila 1) 87654321 (María - Fila 2) 30111222 / 46632600 (Nuevas altas)
Sentencia SQL	Consulta parametrizada sobre Asociados	Nombre de tabla o columna inexistente	Invocación de consulta sobre tabla no creada
Instancia Singleton	Invocación sucesiva de getInstance()	N/A	Comparación directa de referencias de memoria (assertSame)

4.2. Estado Inicial de los Datos Semilla (iniciarBD())

La rutina de inicialización puebla la tabla Asociados con 20 registros iniciales. Los primeros registros relevantes para las aserciones de prueba son:

ID	**Nombre**	**Apellido**	**DNI**	**Calle**	**N.º**	**Ciudad**	**Teléfono**
1	Juan	State	12345678	Buenos Aires	123	Mar del Plata	2235551234
2	Maria	Template	87654321	Córdoba	456	Balcarce	2266555678
3-20	Registros 3 a 20 con nombres de patrones (Observer, Facade, Singleton, etc.)						

4.3. Matriz de Casos de Prueba de Integración (PersistenciaIntegracionTest.java)

La suite se compone de 22 métodos @Test diseñados para verificar todos los contratos de integración de la SRS:

N.º	Método de Prueba	Propósito y Componente Evaluado	Resultado Esperado según SRS
1	testSingletonDevuelveLaMismaInstancia	Patrón Singleton (BaseDeDatos)	Dos llamadas a getInstance() retornan la misma referencia en memoria.
2	testConexionActiva	Verificación de conexión JDBC	Connection.isValid(2) devuelve true.
3	testIniciarBDDejaVeinteRegistrosSemilla	Inicialización de tabla y datos semilla	SELECT COUNT(*) FROM Asociados retorna exactamente 20.
4	testSemillaPrimerRegistroMapeado	Mapeo ResultSet -> DTO	ID 1 devuelve a Juan State con todos sus atributos de calle, ciudad y teléfono.
5	testInsertParametrizado	Inserción directa en BaseDeDatos	Fila total pasa a 21. Se recupera a Nora Nueva (DNI 30111222).
6	testConsultaParametrizadaPorId	Lectura parametrizada BaseDeDatos	SELECT con ? para ID 1 retorna a Juan State en una única fila.
7	testDeleteParametrizadoPorDni	Eliminación directa BaseDeDatos	Baja por DNI 12345678. Total de filas se reduce a 19.
8	testInsertDniDuplicadoLanzaSQLException	Restricción UNIQUE en motor MySQL	Reinsertar DNI 12345678 dispara SQLException por entrada duplicada.
9	testConsultaSqlInvalidaLanzaSQLException	Manejo de excepciones SQL	Ejecutar consulta sobre tabla inexistente lanza SQLException.
10	testGuardarNuevoAsociado	Alta mediante DataAccessObject	Mapeo y guardado de nuevo asociado. Total pasa a 21 filas.
11	testCargarBDDevuelveLaSemilla	Lectura completa DAO	Retorna una lista con exactamente 20 objetos AsociadoDTO.

12	testCargarConLimiteUno	Lectura con límite inferior (1)	Retorna lista de 1 elemento correspondiente a Juan State.
13	testCargarConLimiteCinco	Lectura con límite medio (5)	Retorna lista con los primeros 5 asociados.
14	testCargarConLimiteMayorAlTotal	Límite superior al total (50)	Retorna los 20 asociados existentes sin lanzar excepciones de desbordamiento.
15	testCargarPorIdExistente	Lectura DAO por ID válido	Retorna el DTO de Juan State (DNI 12345678).
16	testCargarPorIdInexistenteDevuelveNull	Lectura DAO por ID fuera de rango	Retorna null sin lanzar excepciones de cursor o SQL.
17	testActualizarModificaElAsociadoIdentificadoPorDni	Modificación por DNI en DAO	Modifica nombre y calle de María (DNI 87654321). Juan (ID 1) permanece inalterado.
18	testEliminarPorDni	Baja DAO por DNI existente	Elimina a Juan (DNI 12345678). Total de filas pasa a 19.
19	testEliminarDniInexistenteNoBorraNada	Baja DAO por DNI inexistente	La base de datos mantiene sus 20 registros sin alterations.
20	testEliminarDniNuloEsRechazado	Precondición de no nulidad	eliminar(null) se rechaza. Mantienen 20 registros.
21	testGuardarDniDuplicadoLanzaSQLException	Integridad relacional vía DAO	Intentar guardar un DTO con DNI 12345678 lanza SQLException.
22	testCrearDTOMapeaElAsociado	Mapeo dominio a DTO en DAO	Mapeo interno asigna correctamente los 7 atributos sin escribir en la BD.

5. Diagnóstico de Defectos y Hallazgos de Integración (Ejecución en ROJO)

5.1 Resultado Inicial sobre el Código Provisto (BUILD FAILURE)

La suite de pruebas de integración se ejecutó sobre la implementación distribuida del DataAccessObject y BaseDeDatos (manteniendo el AsociadoDTO ya refactorizado en la Parte 1). La prueba arrojó un fallo de construcción (BUILD FAILURE), con 19 pruebas aprobadas y 3 pruebas fallidas por errores no capturados.

5.2. Análisis Detallado de los Defectos Descubiertos

```
[INFO] Running persistencia.PersistenciaIntegracionTest
[ERROR] Tests run: 22, Failures: 0, Errors: 3, Skipped: 0 <<< FAILURE! -- in persistencia.PersistenciaIntegracionTest
[ERROR] persistencia.PersistenciaIntegracionTest.testCargarPorIdExistente <<< ERROR!
java.sql.SQLException: Before start of result set
 at persistencia.DataAccessObject.cargar(DataAccessObject.java:88)
[ERROR] persistencia.PersistenciaIntegracionTest.testCargarPorIdInexistenteDevuelveNull <<< ERROR!
java.sql.SQLException: Illegal operation on empty result set.
 at persistencia.DataAccessObject.cargar(DataAccessObject.java:88)
[ERROR] persistencia.PersistenciaIntegracionTest.testActualizarModificaElAsociadoIdentificadoPorDni <<< ERROR!
java.sql.SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'DNI'
 at persistencia.BaseDeDatos.ejecutarActualizacion(BaseDeDatos.java:119)
 at persistencia.DataAccessObject.actualizar(DataAccessObject.java:138)
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
```

Defecto D1: lectura de ResultSet sin Posicionamiento de Cursor (cargar(int))

- ★ *Ubicación:* DataAccessObject.cargar(int id), línea 88.
- ★ *Descripción del Código Original:* Tras ejecutar la consulta SQL, el método accedía directamente a los datos mediante resultado.getInt("id") sin invocar previamente resultado.next().
- ★ *Manifiesto de Fallas:*
  - *Caso ID Existente (id = 1):* Lanza java.sql.SQLException: Before start of result set. El cursor de JDBC se encuentra posicionado antes de la primera fila al crearse el objeto ResultSet. Al no avanzar el cursor con .next(), la lectura falla a pesar de que los datos de Juan State están presentes.
  - *Caso ID Inexistente (id = 999):* Lanza java.sql.SQLException: Illegal operation on empty result set.. Al no haber filas y no comprobar si el conjunto está vacío, el método intenta leer sobre un cursor nulo en lugar de devolver null como especifica la SRS 3.3.
- ★ *Severidad:* Alta. Invalida por completo la funcionalidad de búsqueda individual por identificador.

Defecto D2: Hardcoding del Identificador en Operación de Actualización (actualizar)

- ★ *Ubicación:* DataAccessObject.actualizar y BaseDeDatos.ejecutarActualizacion, línea 119/138.
- ★ *Descripción del Código Original:* DataAccessObject.actualizar construía la sentencia UPDATE ... WHERE id = ?, pero BaseDeDatos.ejecutarActualizacion ignoraba el argumento dinámico del DTO y ejecutaba intencionalmente stmt.setInt(8, 1).
- ★ *Violación de Requisito (SRS 3.3):* La SRS 3.3 exige explícitamente que la actualización y eliminación se identifiquen mediante el número de documento (**DNI**).
- ★ *Manifiesto de Fallas:*
  - Al intentar actualizar los datos de María Template (DNI 87654321, ID 2), el UPDATE apuntó por hardcodeo al registro de Juan State (ID 1), intentando sobreescribir el DNI de Juan con el DNI de María. La base de datos MySQL abortó la transacción lanzando SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'DNI'.
  - *Efecto Oculto:* Si el DNI no hubiera tenido un índice UNIQUE, la operación habría modificado en silencio al registro de Juan State sin alterar a María. La presencia de la restricción de unicidad relacional forzó la manifestación explícita del error.
- ★ *Severidad:* Crítica. Destruye la integridad de los datos en la base relacional al modificar registros erróneos.

5.3. Hallazgos Secundarios de Integración

- ★ *Ausencia de Driver JDBC en Configuración Pom:* El archivo pom.xml original carecía de la dependencia del conector MySQL (mysql-connector-j), provocando que Class.forName("com.mysql.cj.jdbc.Driver") fallara al no encontrar la clase. Se incorporó la versión 8.0.33.
- ★ *Inconsistencia de Mensaje en eliminar(null):* El assert en DataAccessObject.eliminar valida únicamente dni != "", dejando pasar referencias null. No obstante, la prueba 20 aprobó porque BaseDeDatos.ejecutarDelete incluye assert dni != null, deteniendo la ejecución en la capa inferior.

6. Refactorización y Resultados en VERDE

6.1. Cambios en el Código Fuente

Se refactorizaron únicamente los dos métodos señalados por los errores de la suite, manteniendo inalteradas las firmas públicas del API.

- *Corrección del Defecto D1 (DataAccessObject.cargar)*
  Se condicionó el mapeo a la comprobación explícita del avance del cursor mediante if (resultado.next()). Si no existen registros, el método retorna null.

- *Corrección del Defecto D2 (DataAccessObject.actualizar y BaseDeDatos)*
  Se reestructuró la sentencia UPDATE para buscar por el criterio único exigido por la SRS (**DNI**) y se enlazó dinámicamente el parámetro del DNI del asociado.

6.2. Resultados tras la Refactorización (BUILD SUCCESS)

Tras aplicar los cambios en DataAccessObject y BaseDeDatos, se volvió a ejecutar la suite completa de integración (mvn clean test -Dtest=PersistenciaIntegracionTest).
El motor de pruebas confirmó la resolución total de los defectos, alcanzando el estado VERDE:

```
[INFO] --- maven-surefire-plugin:3.2.5:test (default-test) @ Clinica ---
[INFO] Running persistencia.PersistenciaIntegracionTest
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0 -- in
persistencia.PersistenciaIntegracionTest
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

Los 22 casos de prueba de integración fueron completados con éxito, confirmando la estabilidad del módulo de persistencia sobre el motor MySQL / MariaDB.

Parte 3

1. Introducción y Marco de Requisitos

1.1. Componentes Bajo Prueba y Arquitectura del Módulo

La Parte 3 de la estrategia de testing aborda la verificación atómica y de integración sobre el módulo de la ambulancia del sistema clínico. Este módulo combina el patrón de diseño **State** para la gestión del ciclo de vida operacional del vehículo con mecanismos de **concurrencia multihilo** mediante monitores nativos de Java (synchronized, wait() y notifyAll()), el patrón **Observer** para notificar cambios de estado a la interfaz de usuario, y rutinas en segundo plano (RetornoAutomatico).

La especificación exige estructurar las validaciones en dos niveles conceptuales diferenciados:

1. *Pruebas Atómicas con Mocks (AmbulanciaDelegacionMockTest):* Verificación atómica del componente Ambulancia aislado de los estados concretos. Mediante la inyección de un doble de prueba (Mock) de la interfaz State, se comprueba que cada solicitud delegue estrictamente en la función correspondiente de la interfaz sin instanciar DisponibleState ni los demás estados del dominio.
2. *Pruebas de Integración y Concurrencia (AmbulanciaIntegracionConcurrenciaTest):* Verificación dinámica del acoplamiento entre Ambulancia, las seis implementaciones concretas de State y la rutina concurrente RetornoAutomatico, evaluando la correcta exclusión mutua, la sincronización entre hilos y la prevención de bloqueos permanentes (*deadlocks*).

Para evitar la inestabilidad derivada de aleatoriedades (como los intervalos de TiempoMuerto o la selección al azar de servicios por los asociados), las pruebas se diseñaron en forma determinística utilizando únicamente Ambulancia, hilos controlados para los actores (Asociado, Operario), la rutina RetornoAutomatico y sincronización mediante join con tiempo límite (*timeout*).

1.2. Requisitos Operacionales y Criterio de Verdad

A diferencia del módulo de persistencia (regido por la SRS), las reglas de negocio de la ambulancia están definidas por el diseño orientado a objetos, los comentarios Javadoc de las clases y los contratos formales de las librerías base (como java.util.Observable):

- ★ *Transición desde Disponible:* Desde DisponibleState, una solicitud de atención a domicilio, traslado o mantenimiento debe cambiar exitosamente de estado y marcar la ambulancia como ocupada (ocupado = true).
- ★ *Retorno post Traslado:* Al finalizar el traslado de un paciente a la clínica (RetornoClinica), la ambulancia debe regresar a DisponibleState (conforme al comentario explícito en TrasladandoPacienteClinicaState). Bajo ninguna circunstancia debe quedar en RegresandoDelTallerState.
- ★ *Reactivación tras Mantenimiento:* Al retornar del taller (volviendoDelTaller), la ambulancia debe quedar inmediatamente apta para atender una nueva solicitud de domicilio o traslado.
- ★ *Ciclo de Retorno Automático:* La rutina RetornoAutomatico debe permanecer ejecutándose en segundo plano **mientras** isSimulacionActiva() sea true, tal como establece el comentario del método run().

- ★ *Despertar de Hilos en Espera:* Todo hilo que permanezca bloquado en wait() a la espera de la ambulancia debe ser despertado mediante notifyAll() en cuanto el servicio en curso finalice.
- ★ *Contrato del Patrón Observer:* Para que notifyObservers(...) entregue un evento a los observadores registrados en Observable, es condición obligatoria haber ejecutado previamente el método setChanged().

1.3. Entorno de Ejecución y Herramientas de Testing

La suite de pruebas para el módulo de la ambulancia se integró en el esquema de construcción Maven con las siguientes herramientas:

- ★ *Framework de Testing:* JUnit versión 4.13.2.
- ★ *Librería de Mocks:* Mockito versión 4.11.0 (org.mockito:mockito-core), seleccionada por ser la última versión de la rama 4.x compatible con Java 8.
- ★ *Motor de Ejecución:* Maven 3.9.9 con maven-surefire-plugin versión 3.2.5 y aserciones activadas (``).
- ★ *Entorno de Compilación:* JDK 17.0.2 configurado con target Java 8.
- ★ *Suites Implementadas:*
  - src/test/java/modelo/modeloDominio/ambulancia/AmbulanciaDelegacionMockTest.java (Pruebas atómicas con Mocks).
  - src/test/java/modelo/modeloDominio/ambulancia/AmbulanciaIntegracionConcurrenciaTest.java (Pruebas de integración y concurrencia).

Para prevenir bloqueos infinitos (*deadlocks*) durante la ejecución de Maven, cada prueba de concurrencia fue configurada con anotaciones de tiempo máximo de espera @Test(timeout = 2500) y sincronización de hilos vía join(2000). Adicionalmente, el bloque finally de las pruebas asegura la desactivación de la simulación (setSimulacionActiva(false)) y la interrupción explícita de los hilos creados.

2. Diseño Formal de Casos de Prueba

2.1. Suite 1: Delegación Atómica con Mocks (AmbulanciaDelegacionMockTest)

Mediante el uso de Mockito, se reemplaza la instancia de State por un doble de prueba (mock(State.class)), verificando que la clase Ambulancia delegue correctamente el mensaje al estado actual sin invocar métodos no correspondientes.

N.º	**Método de Prueba (@Test)**	**Mensaje Invocado en Ambulancia**	**Verificación de Delegación en State (Mockito)**
1	testAtenderDomicilioDelegaEnElEstado	atenderDomicilio()	Verifica verify(stateMock, times(1)).SolicitudDeAtencionDomicilio() y never() en los demás.
2	testTrasladarDelegaEnElEstado	trasladarALaClinica()	Verifica verify(stateMock, times(1)).SolicitudDeTraslado() y never() en los demás.
3	testSolicitarMantenimientoDelegaEnElEstado	solicitarMantenimiento()	Verifica verify(stateMock, times(1)).SolicitudMantenimiento() y never() en los demás.

4	testRegresarSinPacienteDelegaEnElEstado	regresarSinPaciente()	Verifica verify(stateMock, times(1)).RetornoClinica() y never() en los demás.
5	testRetornoAutomaticoDelegaEnElEstado	retornoAutomatico()	Verifica verify(stateMock, times(1)).RetornoClinica() y never() en los demás.

2.2. Suite 2: Integración de Estados y Concurrencia
(AmbulanciaIntegracionConcurrenciaTest)
Se ejercita la máquina de estados real combinada con la ejecución de hilos concurrentes simulando la llegada simultánea de solicitudes.

N.º	**Método de Prueba (@Test)**	**Escenario / Operación Evaluada**	**Resultado Esperado según Diseño**
1	testDisponibleAceptaDomicilio	Solicitud de domicilio desde DisponibleState.	Pasa a AtendiendoEnDomicilioState, ocupado = true.
2	testDisponibleAceptaTraslado	Solicitud de traslado desde DisponibleState.	Pasa a TrasladandoPacienteClinicaState, ocupado = true.
3	testDisponibleAceptaMantenimiento	Solicitud de mantenimiento desde DisponibleState.	Pasa a EnTallerState, ocupado = true.
4	testTrasladoRegresaADisponibleYNoAlTaller	Invocación a RetornoClinica desde TrasladandoPacienteClinicaState.	Retorna a DisponibleState, ocupado = false.
5	testVolverDelTallerDejaLaAmbulanciaAptaParaAtender	Invocación a volviendoDelTaller() tras mantenimiento.	Retorna a DisponibleState y acepta una nueva atención a domicilio.
6	testRechazoEnTrasladoNotificaAlObservador	Intento de traslado con la ambulancia ya ocupada.	Notifica al Observer registrado el mensaje de rechazo correspondiente.
7	testRetornoAutomaticoSigueVivoMientrasLaSimulacionEstaActiva	Ejecución del hilo RetornoAutomatico con isSimulacionActiva = true.	El hilo permanece en ejecución realizando llamadas periódicas; no termina inmediatamente.
8	testSegundoHiloEsperarHastaLiberacion	Dos hilos compitiendo por la ambulancia ocupada.	El segundo hilo se bloquea en wait() y se despierta tras el notifyAll() del primer hilo.
9	testSegundoHiloNoSeBloqueaDespuesDeUnTraslado	Concurrencia secuencial tras un traslado completo.	El segundo hilo logra atender a domicilio sin quedar trabado en wait().

10	testSegundoHiloNoSeBloqueaDespuesDelTaller	Concurrencia secuencial tras retornar del taller.	El segundo hilo logra solicitar atención sin quedar trabado en wait().
11	testInterrupcionEnWaitTrataExcepcion	Interrupción explícita (interrupt()) de un hilo en wait().	Captura la InterruptedException de forma limpia sin romper el monitor.
12	testWaitNotifyDespiertaAlHiloQueEspera	Verificación explícita del ciclo wait/notifyAll entre dos hilos.	Transición ordenada del segundo hilo al liberar el recurso.

3. Diagnóstico de Defectos y Hallazgos (Ejecución en ROJO)

3.1. Resultado Inicial de la Ejecución (BUILD FAILURE)

La suite se ejecutó sobre el código fuente original del módulo de la ambulancia. El comando de ejecución arrojó el siguiente resumen:

Los 5 casos de delegación atómica con Mockito aprobaron exitosamente, demostrando que la clase Ambulancia invocaba a la interfaz State. Sin embargo, la prueba de integración de estados y concurrencia falló en 6 de sus 11 casos, poniendo al descubierto cuatro defectos críticos de diseño y sincronización.

```
AmbulanciaDelegacionMockTest: Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
AmbulanciaIntegracionConcurrenciaTest: Tests run: 11, Failures: 6, Errors: 0, Skipped: 0
[ERROR] Tests run: 16, Failures: 6, Errors: 0, Skipped: 0 &lt;&lt;&lt; FAILURE!
[INFO] BUILD FAILURE
```

3.2 Diagnóstico Detallado de Defectos Identificados

Defecto D1: Inversión Lógica en el Bucle de RetornoAutomatico

- ★ *Ubicación:* Clase RetornoAutomatico, método run().
- ★ *Código Original:* `while (!ambulancia.isSimulacionActiva()) { ... }`
- ★ *Síntoma:* Dado que el constructor de Ambulancia inicializa simulacionActiva = true, la condición del bucle se evaluaba como false desde el primer instante. El hilo RetornoAutomatico finalizaba su ejecución inmediatamente al arrancar sin realizar ninguna llamada a retornoAutomatico().
- ★ *Falla en Test:* testRetornoAutomaticoSigueVivoMientrasLaSimulacionEstaActiva falló al detectar que el hilo finalizaba de forma prematura.

Defecto D2: Transición Corrupta post Traslado y Retención de Monitor con Thread.sleep

- *Ubicación:* Clase TrasladandoPacienteClinicaState, método RetornoClinica().
- *Código Original:*

```
Thread.sleep(1000);
ambulancia.setState(new RegresandoDelTallerState(ambulancia));
```

- ★ *Síntoma:* Presentaba dos fallas graves:
  1. Al finalizar el traslado de un paciente a la clínica, el estado transicionaba erróneamente a RegresandoDelTallerState en lugar de DisponibleState.
  2. Ejecutaba Thread.sleep(1000) dentro de un método llamado desde un bloque synchronized de Ambulancia. Esto retenía el monitor del objeto durante un

segundo entero, impidiendo que cualquier otro hilo pudiera interactuar con la ambulancia. Además, poseía un bloque catch vacío que absorbía cualquier interrupción de hilo.

- ★ *Falla en Test:* testTrasladoRegresaADisponibleYNoAlTaller falló al comprobar que el estado resultante era RegresandoDelTallerState.

Defecto D3: Bloqueo Permanente de Hilos (*Deadlock*) post Taller / Traslado

- ★ *Ubicación:* Clases RegresandoDelTallerState y Ambulancia.
- ★ *Código Original:* volviendoDelTaller() ejecutaba la transición en EnTallerState, pasando a RegresandoDelTallerState y marcando ocupado = false. Sin embargo, en RegresandoDelTallerState, los métodos SolicitudDeAtencionDomicilio y SolicitudDeTraslado no realizaban ninguna transición válida ni invocaban notifyAll().
- ★ *Síntoma:* Al quedar la ambulancia en RegresandoDelTallerState (ya sea tras volver del taller o tras el traslado corrupto del defecto D2), una nueva solicitud marcaba ocupado = true, pero el estado no cambiaba. Los hilos subsecuentes encontraban ocupado == true y entraban en wait(), quedando bloqueados indefinidamente ya que ningún método ejecutaba notifyAll().
- ★ *Falla en Test:* testSegundoHiloNoSeBloqueaDespuesDeUnTraslado y testSegundoHiloNoSeBloqueaDespuesDelTaller alcanzaron el tiempo límite (*timeout*) demostrando la presencia de un *deadlock*.

Defecto D4: Violación del Contrato de Observable en Notificaciones

- ★ *Ubicación:* Clases de estados concretos (ej. TrasladandoPacienteClinicaState, método SolicitudDeTraslado).
- ★ *Código Original:* Ante una solicitud rechazada, se invocaba directamente a ambulancia.notifyObservers(...) sin ejecutar previamente ambulancia.setChanged().
- ★ *Síntoma:* De acuerdo con la especificación de java.util.Observable, si el indicador de cambio interno no está activado (changed == true), el método notifyObservers() ignora la llamada y no entrega la notificación a los observadores registrados. La interfaz visual de la simulación jamás se enteraba de los rechazos.
- ★ *Falla en Test:* testRechazoEnTrasladoNotificaAlObservador falló al comprobar que el Observer de prueba no recibía el mensaje.

4. Refactorización Realizada

Para solucionar los cuatro defectos detectados sin alterar las firmas públicas del módulo, se aplicaron las siguientes refactorizaciones:

4.1 Corrección de RetornoAutomatico (Defecto D1)

Se corrigió el operador de la condición del bucle en el método run():

4.2 Corrección de Transiciones y Eliminación de sleep en Monitores (Defectos D2 y D3)

Se eliminó el Thread.sleep dentro del método sincronizado de TrasladandoPacienteClinicaState y se corrigió la transición para que retorne a DisponibleState

Asimismo, en RegresandoDelTallerState se implementó la transición adecuada hacia DisponibleState y se aseguró la liberación de la ambulancia con la invocación de setOcupado(false).

4.3 Corrección de Notificaciones a Observadores (Defecto D4)
En todos los métodos de los estados que notifican eventos o rechazos a los observadores, se antepuso la invocación a setChanged().

5. Resultados tras la Refactorización (Ejecución en VERDE)

Una vez aplicadas las correcciones en el módulo de la ambulancia, se re-ejecutó la totalidad de las suites de prueba.

5.1 Resultado de la Ejecución (BUILD SUCCESS)
El motor Surefire reportó la aprobación exitosa del 100% de los casos de prueba, alcanzando el estado **VERDE**:
La suite final incluyó un test adicional (testWaitNotifyDespiertaAlHiloQueEspera), contabilizando **17 métodos de prueba completamente exitosos**.

```
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaDelegacionMockTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0 -- in AmbulanciaDelegacionMockTest
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0 -- in AmbulanciaIntegracionConcurrenciaTest
[INFO] -------------------------------------------------------------------
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] -------------------------------------------------------------------
```

Conclusiones

Conclusión de la Parte 1

En primer lugar, la implementación de una suite unitaria compuesta por únicamente cinco métodos automatizados permitió descubrir dos defectos funcionales severos y un fallo crítico de diseño, sin requerir la puesta en marcha de motores de base de datos ni componentes de interfaz gráfica.

El defecto D1 se manifestaba exclusivamente en el valor de frontera número = 0. Pruebas con datos típicos arbitrarios (como número = 1000) hubieran generado falsos positivos de aprobación, demostrando que el diseño riguroso en los bordes de decisión es indispensable para garantizar la calidad del software.

El hallazgo del defecto D2 demostró que verificar únicamente los métodos de modificación (*setters*) es insuficiente. Es imprescindible evaluar todos los constructores y caminos de entrada de datos a la clase, ya que la lógica de validación puede estar presente en una interfaz y ausente en otra.

Por otro lado, las sentencias assert nativas de Java son herramientas de depuración interna para desarrolladores, no mecanismos de validación de contratos en producción. La sustitución de assert por IllegalArgumentException garantiza que la integridad de los datos se evalúe de forma determinista en cualquier entorno de ejecución de la JVM.

Ante discrepancias entre el código provisto y la documentación de la interfaz gráfica, la Especificación de Requisitos de Software (SRS 3.1) actuó como el único árbitro imparcial para clasificar objetivamente qué comportamiento constituye un defecto del sistema.

Por último, la metodología TDD/BDD de demostración del fallo inicial (ROJO) seguida de la corrección y verificación exitosa (VERDE) proporciona evidencia sobre la capacidad de la suite de pruebas para atrapar errores y servir como red de seguridad contra futuras regresiones.

Conclusión de la Parte 2

Las pruebas unitarias atómicas de la Parte 1 garantizaron que AsociadoDTO fuera internamente consistente en memoria. Sin embargo, solo las pruebas de integración dinámicas con la base de datos real en la Parte 2 fueron capaces de destapar los defectos críticos de infraestructura: la falta de posicionamiento de cursor (ResultSet.next()) y la alteración de registros erróneos por hardcodeo en las sentencias SQL.

La estrategia de reponer el esquema y los 20 registros semilla en el método @Before de la suite de integración permitió ejecutar 22 pruebas destructivas y de modificación sobre MySQL sin que una prueba contaminara el estado de la siguiente, garantizando que las evidencias se puedan comprobar fácilmente.

Por otro lado, la presencia de la restricción de unicidad (UNIQUE) sobre la columna DNI en el esquema relacional actuó como un mecanismo de defensa activo. Al intentar ejecutar una actualización con un ID hardcodeado, la restricción de la base de datos forzó una excepción SQLIntegrityConstraintViolationException, evitando que un bug de código corrupto pasara inadvertido durante las operaciones CRUD.

El reemplazo de sentencias assert por excepciones IllegalArgumentException en el DTO y la correcta parametrización de consultas con PreparedStatement en el DAO consolidan una arquitectura de persistencia robusta, segura contra inyecciones SQL y totalmente alineada con las especificaciones contractuales de la SRS.

Conclusión de la Parte 3

Las pruebas atómicas con Mockito confirmaron que Ambulancia delegaba correctamente las llamadas a la interfaz State (5/5 en verde desde el inicio). Sin embargo, el testing de integración con hilos reales permitió descubrir los defectos de transición y bloqueos (D1 a D4).

Un error de inversión lógica en una guardia (while (!activa) en lugar de while (activa)) anula por completo el funcionamiento de la rutina concurrente RetornoAutomatico.

Por otro lado, transicionar a un estado inadecuado que no gestiona las solicitudes subsecuentes dejaba la ambulancia permanentemente en ocupado = true sin llamar a notifyAll(), transformando una solicitud legítima de un segundo hilo en un *deadlock* indefinido.

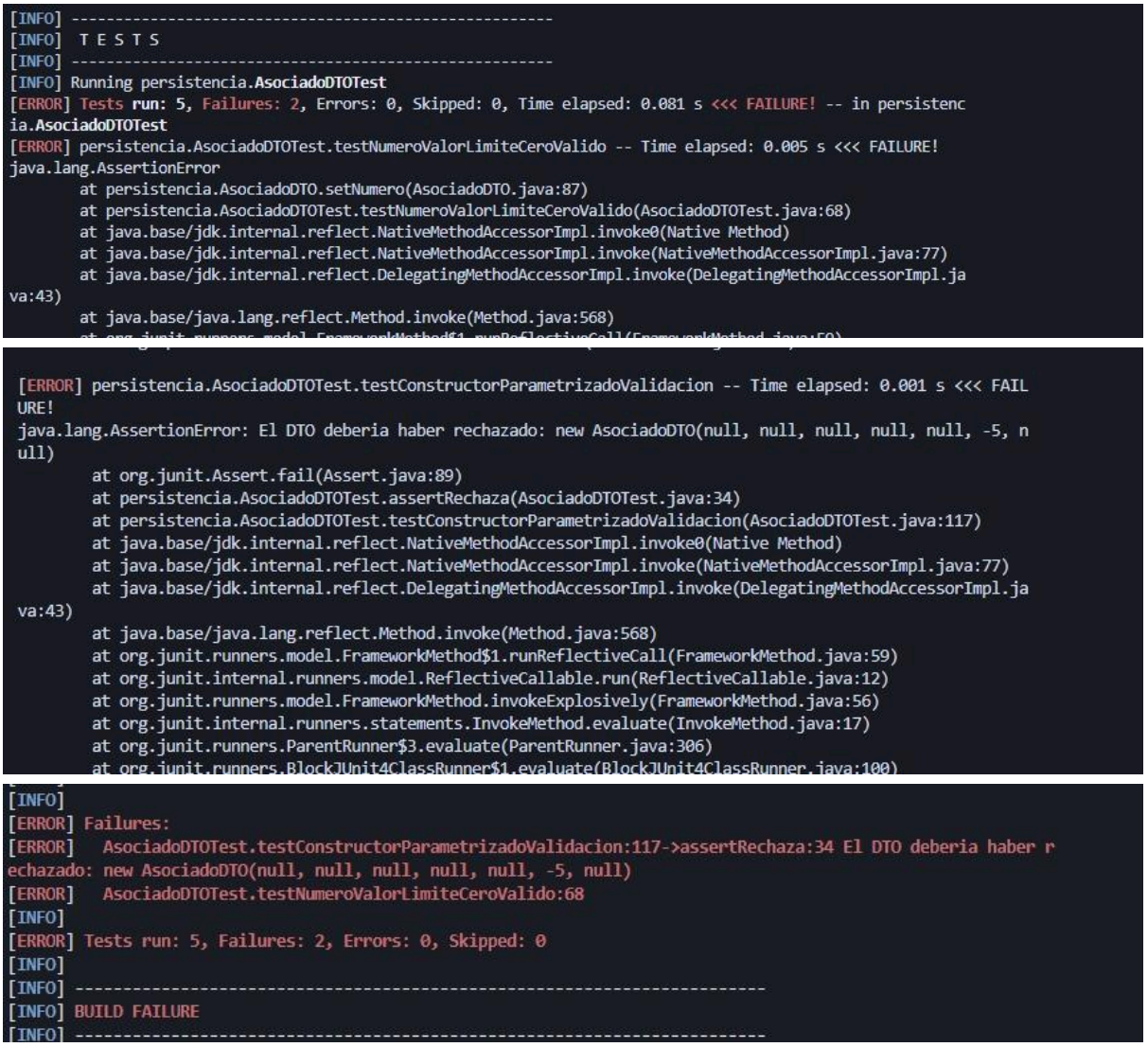
Dormir un hilo dentro de una sección crítica sin liberar el monitor bloquea el acceso de todos los demás procesos al recurso compartido, degradando drásticamente el rendimiento del sistema. También, la omisión de setChanged() impedía la propagación de eventos hacia la interfaz visual. La verificación con observadores de prueba garantizó el desacoplamiento correcto de la arquitectura MVC.

Por último, la utilización de @Test(timeout = 2500) combinada con join y limpieza en bloques finally resultó indispensable para prevenir que fallos de sincronización colgaran indefinidamente la herramienta de integración continua.

Apéndices

Parte 1

Apéndice A.1.1: Traza de Consola en ROJO (BUILD FAILURE)3



```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.AsociadoDTOTest
[ERROR] Tests run: 5, Failures: 2, Errors: 0, Skipped: 0, Time elapsed: 0.081 s <<< FAILURE! -- in persistenc
ia.AsociadoDTOTest
[ERROR] persistencia.AsociadoDTOTest.testNumeroValorLimiteCeroValido -- Time elapsed: 0.005 s <<< FAILURE!
java.lang.AssertionError
        at persistencia.AsociadoDTO.setNumero(AsociadoDTO.java:87)
        at persistencia.AsociadoDTOTest.testNumeroValorLimiteCeroValido(AsociadoDTOTest.java:68)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.ja
va:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
```



```
[ERROR] persistencia.AsociadoDTOTest.testConstructorParametrizadoValidacion -- Time elapsed: 0.001 s <<< FAIL
URE!
java.lang.AssertionError: El DTO deberia haber rechazado: new AsociadoDTO(null, null, null, null, null, -5, n
ull)
        at org.junit.Assert.fail(Assert.java:89)
        at persistencia.AsociadoDTOTest.assertRechaza(AsociadoDTOTest.java:34)
        at persistencia.AsociadoDTOTest.testConstructorParametrizadoValidacion(AsociadoDTOTest.java:117)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.ja
va:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.BlockJUnit4ClassRunner$1.evaluate(BlockJUnit4ClassRunner.java:100)
```



```
[INFO]
[ERROR] Failures:
[ERROR]   AsociadoDTOTest.testConstructorParametrizadoValidacion:117->assertRechaza:34 El DTO deberia haber r
echazado: new AsociadoDTO(null, null, null, null, null, -5, null)
[ERROR]   AsociadoDTOTest.testNumeroValorLimiteCeroValido:68
[INFO]
[ERROR] Tests run: 5, Failures: 2, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
```

Apéndice A.1.2: Traza de Consola en VERDE (BUILD SUCCESS)

```
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ ClinicaManager_TPG ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.AsociadoDTOTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 s -- in persistencia.AsociadoDTOTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.119 s
[INFO] Finished at: 2026-11-05T19:41:28-03:00
```

Parte 2: Trazas de Ejecución de Pruebas de Integración

Apéndice A.2.1 Traza de Ejecución sobre Código Original (ROJO)

```
[ERROR] persistencia.PersistenciaIntegracionTest.testActualizarModificaElAsociadoIdentificadoPorDni -- Time elapsed: 0.046 s <<< ERROR!
java.sql.SQLIntegrityConstraintViolationException: Duplicate entry '87654321' for key 'DNI'
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:118)
        at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeInternal(ClientPreparedStatement.java:916)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdateInternal(ClientPreparedStatement.java:1061)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdateInternal(ClientPreparedStatement.java:1009)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeLargeUpdate(ClientPreparedStatement.java:1320)
        at com.mysql.cj.jdbc.ClientPreparedStatement.executeUpdate(ClientPreparedStatement.java:994)
        at persistencia.BaseDeDatos.ejecutarActualizacion(BaseDeDatos.java:119)
        at persistencia.DataAccessObject.actualizar(DataAccessObject.java:138)
        at persistencia.PersistenciaIntegracionTest.testActualizarModificaElAsociadoIdentificadoPorDni(PersistenciaIntegracionTest.java:216)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.RunBefores.evaluate(RunBefores.java:26)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.BlockJUnit4ClassRunner$1.evaluate(BlockJUnit4ClassRunner.java:100)
        at org.junit.runners.ParentRunner.runLeaf(ParentRunner.java:366)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:103)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:63)
        at org.junit.runners.ParentRunner$4.run(ParentRunner.java:331)
        at org.junit.runners.ParentRunner$1.schedule(ParentRunner.java:79)
        at org.junit.runners.ParentRunner.runChildren(ParentRunner.java:329)
        at org.junit.runners.ParentRunner.access$100(ParentRunner.java:66)
        at org.junit.runners.ParentRunner$2.evaluate(ParentRunner.java:293)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.ParentRunner.run(ParentRunner.java:413)
        at org.apache.maven.surefire.junit4.JUnit4Provider.execute(JUnit4Provider.java:316)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeWithRerun(JUnit4Provider.java:240)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeTestSet(JUnit4Provider.java:214)
        at org.apache.maven.surefire.junit4.JUnit4Provider.invoke(JUnit4Provider.java:155)
        at org.apache.maven.surefire.booter.ForkedBooter.runSuitesInProcess(ForkedBooter.java:385)
        at org.apache.maven.surefire.booter.ForkedBooter.execute(ForkedBooter.java:162)
        at org.apache.maven.surefire.booter.ForkedBooter.run(ForkedBooter.java:507)
        at org.apache.maven.surefire.booter.ForkedBooter.main(ForkedBooter.java:495)
```

```
[ERROR] persistencia.PersistenciaIntegracionTest.testCargarPorIdInexistenteDevuelveNull -- Time elapsed: 0.046 s <<< ERROR!
java.sql.SQLException: Illegal operation on empty result set.
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:130)
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:98)
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:90)
        at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:64)
        at com.mysql.cj.jdbc.result.ResultSetImpl.checkRowPos(ResultSetImpl.java:532)
        at com.mysql.cj.jdbc.result.ResultSetImpl.getObject(ResultSetImpl.java:1322)
        at com.mysql.cj.jdbc.result.ResultSetImpl.getInt(ResultSetImpl.java:830)
        at com.mysql.cj.jdbc.result.ResultSetImpl.getInt(ResultSetImpl.java:851)
        at persistencia.DataAccessObject.cargar(DataAccessObject.java:88)
        at persistencia.PersistenciaIntegracionTest.testCargarPorIdInexistenteDevuelveNull(PersistenciaIntegracionTest.java:208)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.RunBefores.evaluate(RunBefores.java:26)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.BlockJUnit4ClassRunner$1.evaluate(BlockJUnit4ClassRunner.java:100)
        at org.junit.runners.ParentRunner.runLeaf(ParentRunner.java:366)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:103)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:63)
        at org.junit.runners.ParentRunner$4.run(ParentRunner.java:331)
        at org.junit.runners.ParentRunner$1.schedule(ParentRunner.java:79)
        at org.junit.runners.ParentRunner.runChildren(ParentRunner.java:329)
        at org.junit.runners.ParentRunner.access$100(ParentRunner.java:66)
        at org.junit.runners.ParentRunner$2.evaluate(ParentRunner.java:293)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.ParentRunner.run(ParentRunner.java:413)
        at org.apache.maven.surefire.junit4.JUnit4Provider.execute(JUnit4Provider.java:316)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeWithRerun(JUnit4Provider.java:240)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeTestSet(JUnit4Provider.java:214)
        at org.apache.maven.surefire.junit4.JUnit4Provider.invoke(JUnit4Provider.java:155)
        at org.apache.maven.surefire.booter.ForkedBooter.runSuitesInProcess(ForkedBooter.java:385)
        at org.apache.maven.surefire.booter.ForkedBooter.execute(ForkedBooter.java:162)
        at org.apache.maven.surefire.booter.ForkedBooter.run(ForkedBooter.java:507)
        at org.apache.maven.surefire.booter.ForkedBooter.main(ForkedBooter.java:495)
```

```
[INFO]
[INFO] Results:
[INFO]
[ERROR] Errors:
[ERROR]   PersistenciaIntegracionTest.testActualizarModificaElAsociadoIdentificadoPorDni:216 » SQLIntegrityConstraintViolation Duplicate entry '87654321' for k
ey 'DNI'
[ERROR]   PersistenciaIntegracionTest.testCargarPorIdExistente:199 » SQL Before start of result set
[ERROR]   PersistenciaIntegracionTest.testCargarPorIdInexistenteDevuelveNull:208 » SQL Illegal operation on empty result set.
[INFO]
[ERROR] Tests run: 22, Failures: 0, Errors: 3, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.861 s
[INFO] Finished at: 2026-11-06T15:14:25-03:00
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test (default-test) on project ClinicaManager_TPG:
[ERROR]
[ERROR] Please refer to C:\Users\archesan\Desktop\TPE2026\target\surefire-reports for the individual test results.
[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

Apéndice A.2.2 Traza de Ejecución sobre Código Refactorizado (VERDE)

```
C:\Users\archesan\Desktop\TPE2026>"C:\Users\archesan\tools\apache-maven-3.9.9\bin\mvn.cmd" clean test -Dtest=PersistenciaIntegracionTest
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< org.example:ClinicaManager_TPG >-------------------
[INFO] Building ClinicaManager_TPG 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ ClinicaManager_TPG ---
[INFO] Deleting C:\Users\archesan\Desktop\TPE2026\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ ClinicaManager_TPG ---
[INFO] skip non existing resourceDirectory C:\Users\archesan\Desktop\TPE2026\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ ClinicaManager_TPG ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 73 source files with javac [debug target 8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/main/java/modelo/modeloDominio/personas/PersonaObservable.java: Some input files use or override a deprecated API.
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/main/java/modelo/modeloDominio/personas/PersonaObservable.java: Recompile with -Xlint:deprecation for details.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ ClinicaManager_TPG ---
[INFO] skip non existing resourceDirectory C:\Users\archesan\Desktop\TPE2026\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ ClinicaManager_TPG ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug target 8] to target\test-classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ ClinicaManager_TPG ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running persistencia.PersistenciaIntegracionTest
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.366 s -- in persistencia.PersistenciaIntegracionTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.347 s
[INFO] Finished at: 2026-11-06T15:18:44-03:00
[INFO] ------------------------------------------------------------------------
```

Parte 3: Evidencias de Ejecución

Apéndice A.3.1 Evidencia de Ejecución sobre Código Original (ROJO)

```
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testRechazoEnTrasladoNotificaAlObservador -- Time elapsed: 0.002 s <<< FAILURE!
java.lang.AssertionError: notifyObservers sin setChanged no entrega el rechazo
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.assertTrue(Assert.java:42)
        at org.junit.Assert.assertFalse(Assert.java:65)
        at modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testRechazoEnTrasladoNotificaAlObservador(AmbulanciaIntegracionConcurrenciaTes
t.java:124)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.RunBefores.evaluate(RunBefores.java:26)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.BlockJUnit4ClassRunner$1.evaluate(BlockJUnit4ClassRunner.java:100)
        at org.junit.runners.ParentRunner.runLeaf(ParentRunner.java:366)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:103)
        at org.junit.runners.BlockJUnit4ClassRunner.runChild(BlockJUnit4ClassRunner.java:63)
        at org.junit.runners.ParentRunner$4.run(ParentRunner.java:331)
        at org.junit.runners.ParentRunner$1.schedule(ParentRunner.java:79)
        at org.junit.runners.ParentRunner.runChildren(ParentRunner.java:329)
        at org.junit.runners.ParentRunner.access$100(ParentRunner.java:66)
        at org.junit.runners.ParentRunner$2.evaluate(ParentRunner.java:293)
        at org.junit.runners.ParentRunner$3.evaluate(ParentRunner.java:306)
        at org.junit.runners.ParentRunner.run(ParentRunner.java:413)
        at org.apache.maven.surefire.junit4.JUnit4Provider.execute(JUnit4Provider.java:316)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeWithRerun(JUnit4Provider.java:240)
        at org.apache.maven.surefire.junit4.JUnit4Provider.executeTestSet(JUnit4Provider.java:214)
        at org.apache.maven.surefire.junit4.JUnit4Provider.invoke(JUnit4Provider.java:155)
        at org.apache.maven.surefire.booter.ForkedBooter.runSuitesInProcess(ForkedBooter.java:385)
        at org.apache.maven.surefire.booter.ForkedBooter.execute(ForkedBooter.java:162)
        at org.apache.maven.surefire.booter.ForkedBooter.run(ForkedBooter.java:507)
        at org.apache.maven.surefire.booter.ForkedBooter.main(ForkedBooter.java:495)
```

```
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testRetornoAutomaticoSigueVivoMientrasLaSimulacionEstaActiva -- Time elapsed: 0.0
02 s <<< FAILURE!
java.lang.AssertionError: RetornoAutomatico no debe terminar apenas arranca si la simulacion esta activa
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.assertTrue(Assert.java:42)
        at modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testRetornoAutomaticoSigueVivoMientrasLaSimulacionEstaActiva(AmbulanciaIntegra
cionConcurrenciaTest.java:138)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:299)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:293)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.lang.Thread.run(Thread.java:833)

[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testSegundoHiloNoSeBloqueaDespuesDeUnTraslado -- Time elapsed: 3.527 s <<< FAILUR
E!
java.lang.AssertionError: El segundo hilo quedo en wait() porque el estado posterior al traslado no libera el recurso
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.assertTrue(Assert.java:42)
        at org.junit.Assert.assertFalse(Assert.java:65)
        at modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testSegundoHiloNoSeBloqueaDespuesDeUnTraslado(AmbulanciaIntegracionConcurrenci
aTest.java:163)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:299)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:293)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.lang.Thread.run(Thread.java:833)
```

```
[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testSegundoHiloNoSeBloqueaDespuesDelTaller -- Time elapsed: 2.014 s <<< FAILURE!
java.lang.AssertionError: El segundo hilo quedo en wait() despues de volver del taller
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.assertTrue(Assert.java:42)
        at org.junit.Assert.assertFalse(Assert.java:65)
        at modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testSegundoHiloNoSeBloqueaDespuesDelTaller(AmbulanciaIntegracionConcurrenciaTe
st.java:188)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:299)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:293)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.lang.Thread.run(Thread.java:833)

[ERROR] modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testTrasladoRegresaADisponibleYNoAlTaller -- Time elapsed: 1.012 s <<< FAILURE!
java.lang.AssertionError: Tras dejar al paciente en la clinica no debe quedar regresando del taller
        at org.junit.Assert.fail(Assert.java:89)
        at org.junit.Assert.assertTrue(Assert.java:42)
        at org.junit.Assert.assertFalse(Assert.java:65)
        at modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest.testTrasladoRegresaADisponibleYNoAlTaller(AmbulanciaIntegracionConcurrenciaTes
t.java:100)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:59)
        at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
        at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:56)
        at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:299)
        at org.junit.internal.runners.statements.FailOnTimeout$CallableStatement.call(FailOnTimeout.java:293)
        at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:264)
        at java.base/java.lang.Thread.run(Thread.java:833)
```

Apendice A.3.2 Evidencia de Ejecución sobre Código Refactorizado (VERDE)

```
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ ClinicaManager_TPG ---
[INFO] Deleting C:\Users\archesan\Desktop\TPE2026\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ ClinicaManager_TPG ---
[INFO] skip non existing resourceDirectory C:\Users\archesan\Desktop\TPE2026\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ ClinicaManager_TPG ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 73 source files with javac [debug target 8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/main/java/modelo/modeloDominio/personas/PersonaObservable.java: Some input files use or override a deprecated API.
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/main/java/modelo/modeloDominio/personas/PersonaObservable.java: Recompile with -Xlint:deprecation for details.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ ClinicaManager_TPG ---
[INFO] skip non existing resourceDirectory C:\Users\archesan\Desktop\TPE2026\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ ClinicaManager_TPG ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 4 source files with javac [debug target 8] to target\test-classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/test/java/modelo/modeloDominio/ambulancia/AmbulanciaIntegracionConcurrenciaTest.java: C:\Users\archesan\Desktop\TPE2026\src\test\java\modelo\modeloDominio\ambulancia\AmbulanciaInte
gracionConcurrenciaTest.java uses or overrides a deprecated API.
[INFO] /C:/Users/archesan/Desktop/TPE2026/src/test/java/modelo/modeloDominio/ambulancia/AmbulanciaIntegracionConcurrenciaTest.java: Recompile with -Xlint:deprecation for details.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ ClinicaManager_TPG ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaDelegacionMockTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.836 s -- in modelo.modeloDominio.ambulancia.AmbulanciaDelegacionMockTest
[INFO] Running modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.575 s -- in modelo.modeloDominio.ambulancia.AmbulanciaIntegracionConcurrenciaTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.728 s
[INFO] Finished at: 2026-11-06T16:21:03-03:00
```